

Salient Object Detection Using Deep Learning and PyTorch

Abstract

This project presents the development of a Salient Object Detection (SOD) system using Deep Learning and PyTorch. The aim of the project is to automatically detect and segment the most visually important object in an image. The model was built from scratch using a CNN encoder-decoder architecture and trained on the DUTS dataset. The implementation included data preprocessing, augmentation, model training, evaluation, visualization of predictions, and an interactive Google Colab demo. The final model achieved approximately 70% IoU, 78% Precision, 86% Recall, and 79% F1 Score. The project demonstrates the complete workflow of building and evaluating a deep learning segmentation system.

Chapter 1: Introduction

1.1 Background

Computer Vision is an important field of Artificial Intelligence that enables computers to analyze and understand images and videos. One major challenge in computer vision is identifying the most important object in an image.

Salient Object Detection (SOD) focuses on detecting and segmenting the visually significant object in a scene. For example, in an image containing a person or an animal, the model identifies that object as the main focus of attention.

SOD has many real-world applications such as image segmentation, autonomous vehicles, robotics, medical imaging, video surveillance, image editing, and augmented reality.

Chapter 2: Project Objectives

2.1 Main Objective

The main objective of this project is to design and implement a Salient Object Detection model using PyTorch capable of accurately predicting salient object masks from input images.

2.2 Specific Objectives

The project includes the following objectives:

1. Build a dataset loading pipeline.
2. Preprocess and augment image data.
3. Implement a CNN encoder-decoder architecture.
4. Train the model using a segmentation loss function.
5. Evaluate the model using segmentation metrics.
6. Visualize prediction results.
7. Create a simple Google Colab demo.

8. Achieve high IoU and F1 score performance.

Chapter 3: Technologies and Tools Used

3.1 Python

Python was used as the main programming language for this project because it is one of the most popular languages in Artificial Intelligence and Machine Learning. Python provides simple syntax, powerful libraries, and strong community support, making it suitable for deep learning applications.

3.2 PyTorch

PyTorch was used as the main deep learning framework to build and train the neural network model. It provides tensor operations, automatic differentiation, GPU acceleration, and neural network modules that simplify the implementation of deep learning systems.

3.3 Google Colab

Google Colab was used for training and experimentation because it provides free cloud-based GPU resources. It also allows easy notebook sharing, integration with Google Drive, and faster model training compared to CPU-only environments.

3.4 Google Drive

Google Drive was used to store the dataset, trained models, checkpoints, visualizations, and project results. It also helped synchronize files between Google Colab and the local system.

Chapter 4: Dataset

4.1 DUTS Dataset

This project uses the DUTS dataset, which is one of the most popular datasets for Salient Object Detection tasks. The dataset contains thousands of natural images together with corresponding binary masks that identify the salient objects. It includes different object categories, complex backgrounds, and high-quality annotations, making it suitable for training deep learning segmentation models.

The DUTS dataset is divided into two parts:

DUTS-TR for training

DUTS-TE for testing

4.2 Dataset Structure

The dataset was organized into two main folders: one containing the images and another containing the corresponding masks. Each image had a matching mask with the same filename, allowing the model to learn the relationship between input images and segmentation masks.

4.3 Dataset Size

The final dataset used in this project contained 10,553 images and 10,553 masks. The dataset was automatically divided into training, validation, and testing subsets. Approximately 70% of the data was used for training, 15% for validation, and 15% for testing.

The final split included:

7,387 training samples

1,582 validation samples

1,584

testing

samples

```
Split -> Train: 7387 | Val: 1582 | Test: 1584
Image batch: torch.Size([8, 3, 224, 224]) range: 0.0 1.0
Mask batch : torch.Size([8, 1, 224, 224]) range: 0.0 1.0
```

4.4 Data Preprocessing

Before training, all images were preprocessed using resizing, tensor conversion, normalization, and data augmentation. The final image size used in the project was 224×224 pixels.

To improve generalization and reduce overfitting, augmentation techniques such as horizontal flipping, brightness adjustment, and random cropping were applied during training.

```
[ ] import os

print("Images:", len(os.listdir("/content/drive/MyDrive/sod_project/dataset/images")))
print("Masks:", len(os.listdir("/content/drive/MyDrive/sod_project/dataset/masks")))

Images: 10553
Masks: 10553
```

Chapter 5: Model Architecture

5.1 Encoder-Decoder Architecture

The project uses a CNN encoder-decoder architecture for salient object segmentation. The model consists of an encoder, bottleneck, decoder, and output layer.

The encoder extracts important visual features from the input image using convolutional layers, Batch Normalization, ReLU activation, and MaxPooling. The decoder reconstructs the segmentation mask using transposed convolutions and upsampling layers.

Batch Normalization was added to improve training stability and convergence speed, while Dropout regularization helped reduce overfitting. The final output layer uses Sigmoid activation to generate pixel values between 0 and 1, representing foreground and background probabilities.

```
Input : torch.Size([2, 3, 224, 224])
Output: torch.Size([2, 1, 224, 224]) range: 0.12821383774280548 0.8268979787826538
Total parameters: 1,154,785
```

Chapter 6: Loss Function

6.1 BCE Loss

Binary Cross Entropy (BCE) Loss measures pixel-wise classification error.

BCE compares predicted probabilities with ground truth masks.

6.2 IoU Loss

Intersection over Union (IoU) Loss measures overlap quality between predicted and true masks.

IoU is defined as:

- Intersection / Union

Higher IoU indicates better segmentation performance.

6.3 Combined Loss

The project combines BCE and IoU losses:

$$\text{Loss} = \text{BCE} + 0.5 * (1 - \text{IoU})$$

Advantages of combined loss:

- Better segmentation quality
- Improved boundary prediction
- More stable training

Chapter 7: Training Process

```
Epoch 001/25 | Train: 0.7889 | Val: 0.7006 | IoU: 0.4996 | P: 0.6437 | R: 0.7331 | F1: 0.6387
✓ New best! Saved -> /content/drive/MyDrive/sod_project/checkpoints/best_model.pt

Epoch 002/25 | Train: 0.7195 | Val: 0.7352 | IoU: 0.5242 | P: 0.5895 | R: 0.8692 | F1: 0.6584
No improvement for 1 epoch(s).

Epoch 003/25 | Train: 0.6805 | Val: 0.6370 | IoU: 0.5279 | P: 0.7221 | R: 0.6899 | F1: 0.6592
✓ New best! Saved -> /content/drive/MyDrive/sod_project/checkpoints/best_model.pt

*** Epoch 008/25 | Train: 0.5710 | Val: 0.5440 | IoU: 0.6116 | P: 0.7453 | R: 0.7919 | F1: 0.7335
No improvement for 1 epoch(s).

Epoch 009/25 | Train: 0.5583 | Val: 0.5517 | IoU: 0.6212 | P: 0.6868 | R: 0.8879 | F1: 0.7407
No improvement for 2 epoch(s).

Epoch 010/25 | Train: 0.5464 | Val: 0.5290 | IoU: 0.6339 | P: 0.7119 | R: 0.8722 | F1: 0.7511
✓ New best! Saved -> /content/drive/MyDrive/sod_project/checkpoints/best_model.pt

Epoch 011/25 | Train: 0.5358 | Val: 0.5134 | IoU: 0.6468 | P: 0.7304 | R: 0.8717 | F1: 0.7606
✓ New best! Saved -> /content/drive/MyDrive/sod_project/checkpoints/best_model.pt

Epoch 016/25 | Train: 0.4970 | Val: 0.4825 | IoU: 0.6500 | P: 0.7840 | R: 0.8063 | F1: 0.7639
✓ New best! Saved -> /content/drive/MyDrive/sod_project/checkpoints/best_model.pt

Epoch 017/25 | Train: 0.4925 | Val: 0.4941 | IoU: 0.6565 | P: 0.7388 | R: 0.8703 | F1: 0.7686
No improvement for 1 epoch(s).

Epoch 018/25 | Train: 0.4890 | Val: 0.4982 | IoU: 0.6413 | P: 0.7690 | R: 0.8098 | F1: 0.7569
No improvement for 2 epoch(s).

Epoch 019/25 | Train: 0.4833 | Val: 0.4746 | IoU: 0.6603 | P: 0.7756 | R: 0.8285 | F1: 0.7716
✓ New best! Saved -> /content/drive/MyDrive/sod_project/checkpoints/best_model.pt

Epoch 004/25 | Train: 0.6516 | Val: 0.7017 | IoU: 0.4538 | P: 0.7504 | R: 0.5643 | F1: 0.5900
No improvement for 1 epoch(s).

Epoch 005/25 | Train: 0.6282 | Val: 0.5849 | IoU: 0.5804 | P: 0.7065 | R: 0.7911 | F1: 0.7092
✓ New best! Saved -> /content/drive/MyDrive/sod_project/checkpoints/best_model.pt

Epoch 006/25 | Train: 0.6066 | Val: 0.5622 | IoU: 0.6057 | P: 0.6975 | R: 0.8470 | F1: 0.7288
✓ New best! Saved -> /content/drive/MyDrive/sod_project/checkpoints/best_model.pt

Epoch 007/25 | Train: 0.5868 | Val: 0.5357 | IoU: 0.6208 | P: 0.7369 | R: 0.8203 | F1: 0.7424
✓ New best! Saved -> /content/drive/MyDrive/sod_project/checkpoints/best_model.pt

Epoch 012/25 | Train: 0.5261 | Val: 0.4947 | IoU: 0.6514 | P: 0.7628 | R: 0.8335 | F1: 0.7660
✓ New best! Saved -> /content/drive/MyDrive/sod_project/checkpoints/best_model.pt

Epoch 013/25 | Train: 0.5230 | Val: 0.4977 | IoU: 0.6518 | P: 0.7403 | R: 0.8649 | F1: 0.7653
No improvement for 1 epoch(s).

Epoch 014/25 | Train: 0.5120 | Val: 0.4988 | IoU: 0.6462 | P: 0.7442 | R: 0.8474 | F1: 0.7606
No improvement for 2 epoch(s).

Epoch 015/25 | Train: 0.5022 | Val: 0.4832 | IoU: 0.6539 | P: 0.7740 | R: 0.8235 | F1: 0.7675
✓ New best! Saved -> /content/drive/MyDrive/sod_project/checkpoints/best_model.pt

*** Epoch 021/25 | Train: 0.4726 | Val: 0.5225 | IoU: 0.6395 | P: 0.7047 | R: 0.8875 | F1: 0.7544
No improvement for 2 epoch(s).

Epoch 022/25 | Train: 0.4675 | Val: 0.4576 | IoU: 0.6787 | P: 0.7694 | R: 0.8650 | F1: 0.7861
✓ New best! Saved -> /content/drive/MyDrive/sod_project/checkpoints/best_model.pt

Epoch 023/25 | Train: 0.4617 | Val: 0.4779 | IoU: 0.6600 | P: 0.7442 | R: 0.8677 | F1: 0.7711
No improvement for 1 epoch(s).

Epoch 024/25 | Train: 0.4604 | Val: 0.4739 | IoU: 0.6766 | P: 0.7433 | R: 0.8965 | F1: 0.7847
No improvement for 2 epoch(s).

Epoch 025/25 | Train: 0.4554 | Val: 0.4822 | IoU: 0.6709 | P: 0.7360 | R: 0.8970 | F1: 0.7792
No improvement for 3 epoch(s).

Training complete. Best validation loss: 0.4576
Best model saved to: /content/drive/MyDrive/sod_project/checkpoints/best_model.pt
```

7.1 Training Environment

Training was performed using Google Colab with GPU acceleration.

GPU usage significantly reduced training time.

7.2 Hyperparameters

The following hyperparameters were used:

Parameter	Value
Image Size	224x224
Batch Size	8
Epochs	25
Learning Rate	0.0005
Optimizer	Adam

Parameter	Value
Patience	6

7.3 Optimizer

The Adam optimizer was used because it combines:

- Momentum
- Adaptive learning rates

Advantages:

- Faster convergence
- Stable training
- Good performance in deep learning tasks

7.4 Early Stopping

Early stopping was implemented to avoid overfitting. Training stops when validation loss stops improving.

7.5 Checkpoint Saving

The project saves:

- Best model
- Last checkpoint

This allows training continuation and best-model restoration.

7.6 Training Results

During training:

- Training loss decreased steadily
- Validation performance improved
- IoU and F1 scores increased over epochs

The final model achieved strong segmentation performance.

Chapter 8: Evaluation Metrics

8.1 Intersection over Union (IoU)

IoU measures overlap between predicted and true masks.

Formula:

$$\text{IoU} = \text{Intersection} / \text{Union}$$

Higher IoU means better segmentation quality.

Final IoU:

70.98%

```
... Loaded best checkpoint from epoch 22.  
Evaluating on test set: 100%  198/198 [00:18<00:00, 11.69it/s]  
  
===== Test Set Results =====  
Samples evaluated : 1584  
IoU      : 0.7098  
Precision : 0.7757  
Recall    : 0.8593  
F1 Score  : 0.7872  
=====
```

8.2 Precision

Precision measures how many predicted salient pixels are correct.

Formula:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Final Precision:

77.57%

8.3 Recall

Recall measures how many true salient pixels were detected.

Formula:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Final Recall:

85.93%

8.4 F1 Score

F1 Score balances Precision and Recall.

Formula:

$$\text{F1} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

Final F1 Score:

78.72%

Chapter 9: Visualization and Demo

9.1 Visualization

The project includes visualization of:

- Input image

- Ground truth mask
- Predicted mask
- Overlay visualization

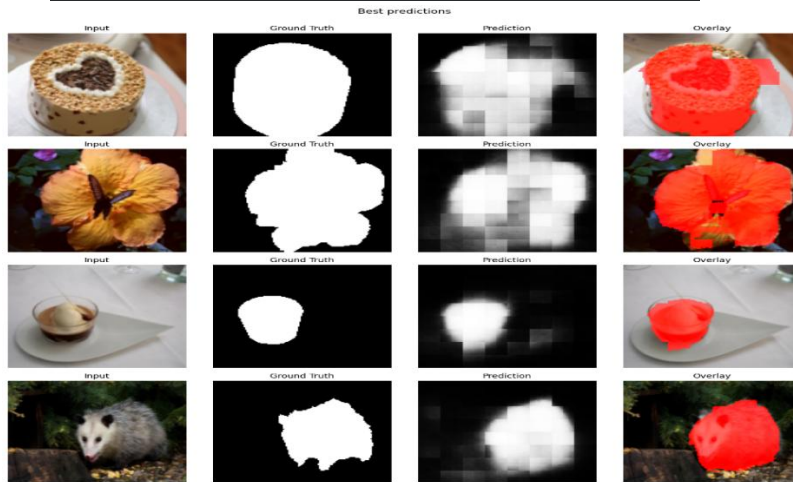
Visualization helps evaluate qualitative model performance.

9.2 Best Predictions

Best predictions demonstrated:

- Accurate object localization
- Strong boundary segmentation
- Good foreground-background separation

```
***
Best predictions (IoUs: [0.855, 0.854, 0.833, 0.819])
Saved: /content/drive/MyDrive/sod_project/results/best_predictions.png
```

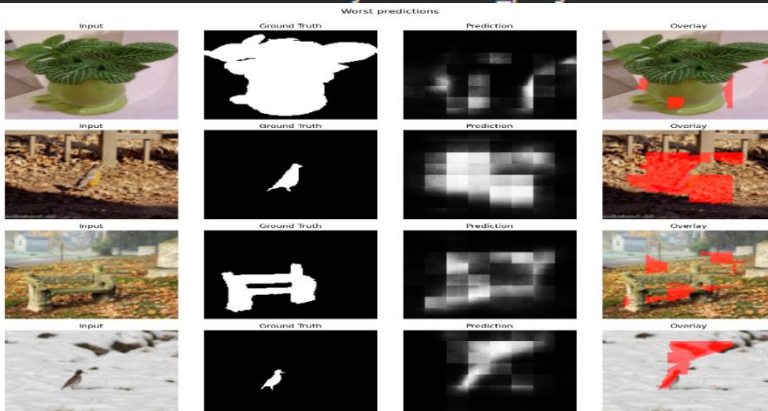


9.3 Worst Predictions

Worst predictions occurred in:

- Complex backgrounds
- Low contrast scenes
- Multiple salient objects

```
Worst predictions (IoUs: [0.024, 0.084, 0.086, 0.089])
Saved: /content/drive/MyDrive/sod_project/results/worst_predictions.png
```



9.4 Google Colab Demo

A Google Colab demo notebook was created for project presentation.

The demo allows:

1. Uploading an image
2. Running model inference
3. Displaying prediction results

The demo produces:

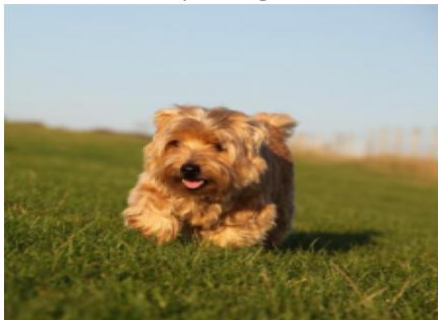
- Original image
- Predicted mask
- Overlay output

This makes the project interactive and presentation-ready.

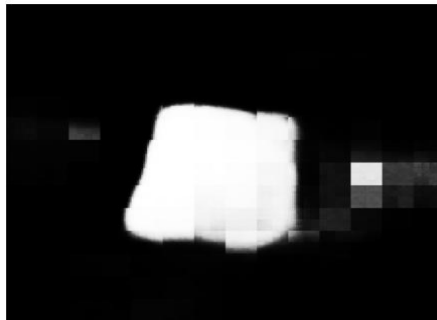
```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

✓ Model loaded.

Input image



Inference time: 43.4 ms
Predicted mask



Overlay



Chapter 10: Challenges Faced

During the development of this project, several challenges were encountered. Managing more than 10,000 images and masks required efficient dataset organization and storage management using Google Drive. Training with high-resolution images also required significant computation time, while some DataLoader multiprocessing issues were solved by setting `NUM_WORKERS = 0`. To reduce overfitting, techniques such as Dropout, data augmentation, validation monitoring, and early stopping were applied.

The developed system has several advantages. It was built completely from scratch using deep learning segmentation techniques and supports GPU acceleration for faster training. The project also includes interactive visualization and a modular structure that makes the system easier to improve and extend in the future.

However, the project also has some limitations. The segmentation performance depends heavily on dataset quality, and training with high-resolution images requires more GPU memory. In addition, complex scenes with multiple objects or difficult backgrounds remain challenging for the model.

Several future improvements could further increase performance and usability. More advanced architectures such as U-Net with skip connections or attention mechanisms could improve segmentation quality. Pretrained backbones like ResNet or EfficientNet may also increase accuracy. In the future, the system could be optimized for real-time applications and deployed as a web or mobile application.

Chapter 13: Conclusion

This project successfully implemented a Salient Object Detection system using Deep Learning and PyTorch. The model was trained on the DUTS dataset and achieved strong segmentation results, including approximately 70% IoU, 78% Precision, 86% Recall, and 79% F1 Score.

The project covered the complete workflow of dataset preparation, model training, evaluation, visualization, and demo creation. It also demonstrated practical knowledge in Computer Vision, Deep Learning, Image Segmentation, and GPU-based training.